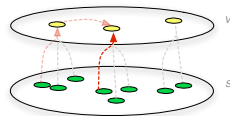


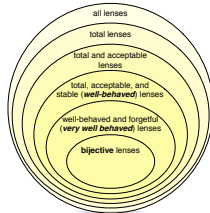
Semantics of Bidirectional Languages



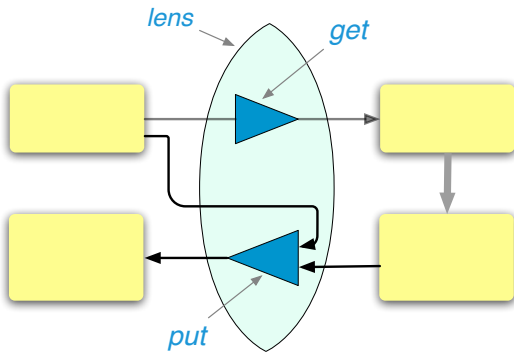
Nate Foster
University of Pennsylvania



GRACE-BX '08



Bidirectional Languages

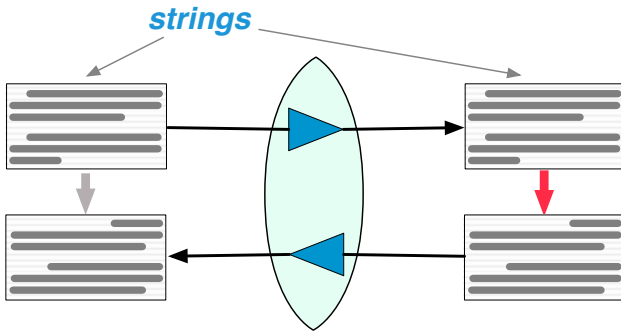


Eliminates Redundancy: programs describes two functions

Ensures Correctness: type system guarantees well-behavedness



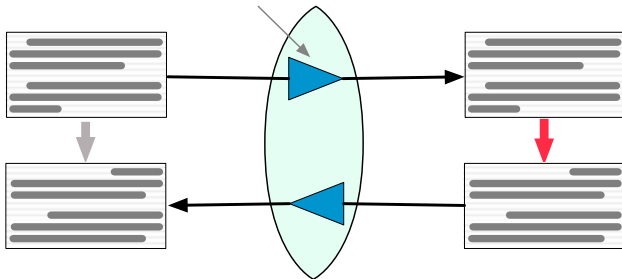
Boomerang [POPL '08]





Boomerang [POPL '08]

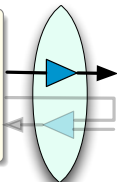
*based on
finite-state transducers*



Main application: heterogeneous data synchronization.

Example: MediaWiki (Get)

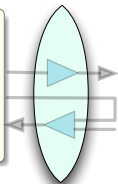
```
==Chefs==  
* Julia Child  
==Justices==  
* Arthur Goldberg
```



```
<html>  
  <body>  
    <h2>Chefs</h2>  
    <ul>  
      <li>Julia Child</li>  
    </ul>  
    <h2>Justices</h2>  
    <ul>  
      <li>Arthur Goldberg</li>  
    </ul>  
  </body>  
</html>
```

Example: MediaWiki (Update)

==Chefs==
* Julia Child
==Justices==
* Arthur Goldberg



<html>
<html>
<body>
 <h2>Chefs</h2>

 Julia Child
 Jacques Pepin

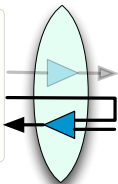
 <h2>Justices</h2>

 Warren Burger
 Arthur Goldberg

</body>
</html>

Example: MediaWiki (Put)

```
==Chefs==  
* Julia Child  
* Jacques Pepin  
==Justices==  
* Warren Burger  
* Arthur Goldberg
```



```
<html>  
<html>  
  <body>  
    <h2>Chefs</h2>  
    <ul>  
      <li>Julia Child</li>  
      <li>Jacques Pepin</li>  
    </ul>  
    <h2>Justices</h2>  
    <ul>  
      <li>Warren Burger</li>  
      <li>Arthur Goldberg</li>  
    </ul>  
  </body>  
</html>
```

Example: MediaWiki (Lens)

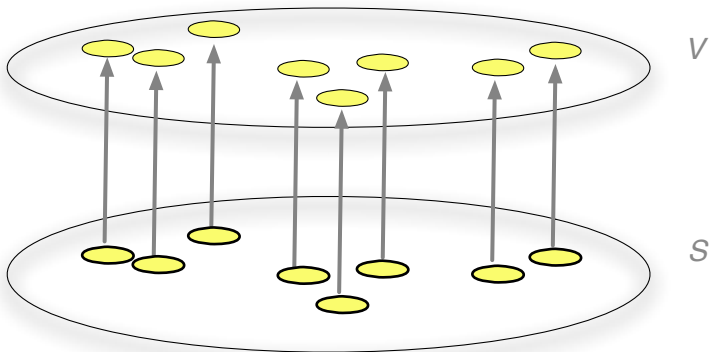
```
(* helpers *)
let mk_elt (ws:string) (tag:string) (body:lens) = ...
let mk_simple_elt (ws:string) (tag:string) (body:lens) =
  ins ws .
  ins ("<" . tag . ">") .
  body .
  ins ("</" . tag . ">")

(* main lenses *)
let p : lens =
  mk_simple_elt nl4 "p" ((text . nl)* . (text . del nl))
let li : lens =
  mk_simple_elt nl6 "li" (del "*" . text)
let ul : lens =
  mk_elt nl4 "ul" (li . del nl)+
let h2 : lens =
  mk_simple_elt nl4 "h2" (del "==" . text . del "==")
let s : lens =
  (del nl . (p | ul))*
let html : lens =
  mk_outer_elt nl0 "html" (mk_elt nl2 "body" s*)
```

Semantics-Driven Language Design

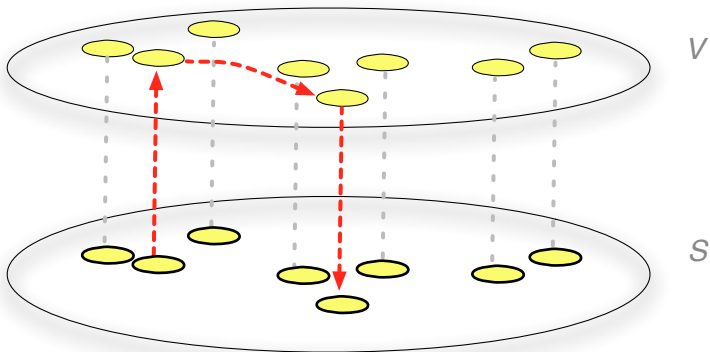
Bijjective Lenses

Bijjective transformations are a simple case...



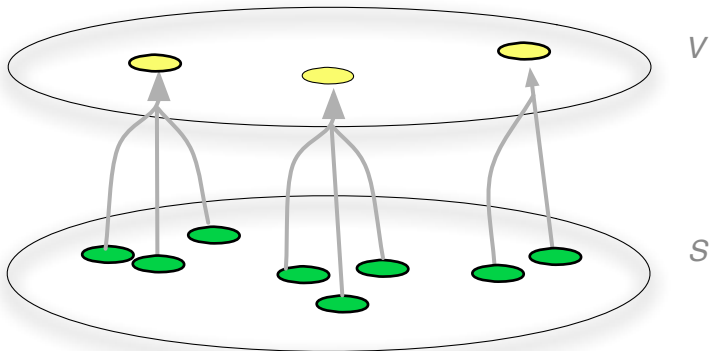
Bijjective Lenses

Bijjective transformations are a simple case...



From Bijective to Bidirectional

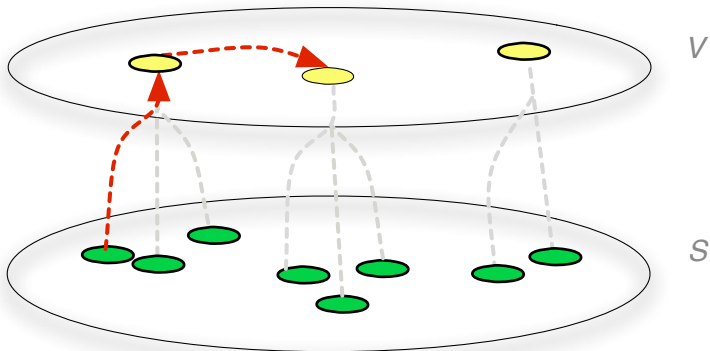
But bijectiveness is a **strong restriction**. Often the whole point of defining a “view” is to **hide** some information in the source! ...But then we need different constraints on *get* and *put*.



Acceptability (PUTGET)

Principle: Updates should be “translated exactly”—i.e., to a source for which *get* yields exactly the updated view.

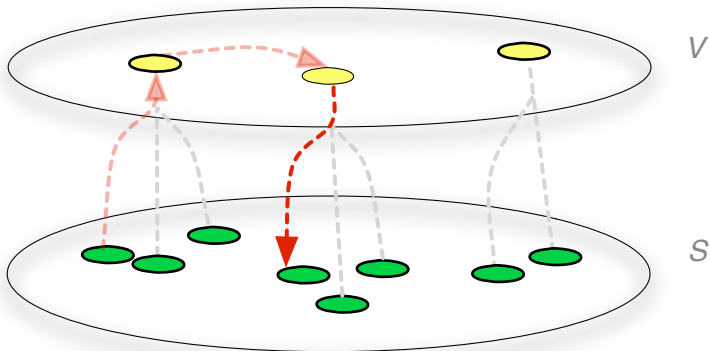
$$\textit{get}(\textit{put } v \textit{ } s) = v$$



Acceptability (PUTGET)

Principle: Updates should be “translated exactly”—i.e., to a source for which *get* yields exactly the updated view.

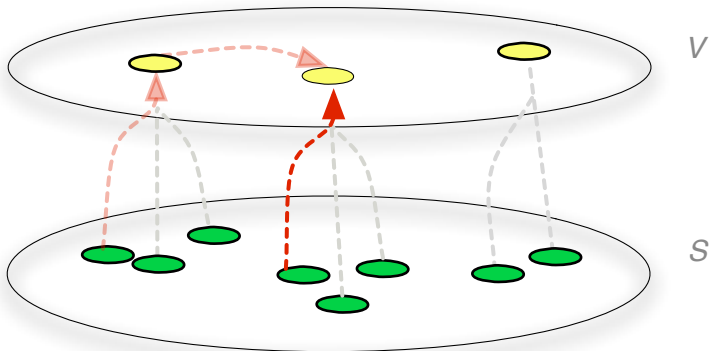
$$\textit{get}(\textit{put } v \textit{ } s) = v$$



Acceptability (PUTGET)

Principle: Updates should be “translated exactly”—i.e., to a source for which *get* yields exactly the updated view.

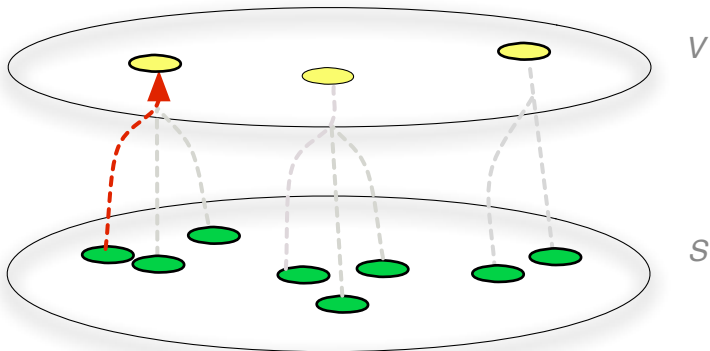
$$\textit{get}(\textit{put } v \textit{ } s) = v$$



Stability (GETPUT)

Principle: If the view does not change, then neither should the source.

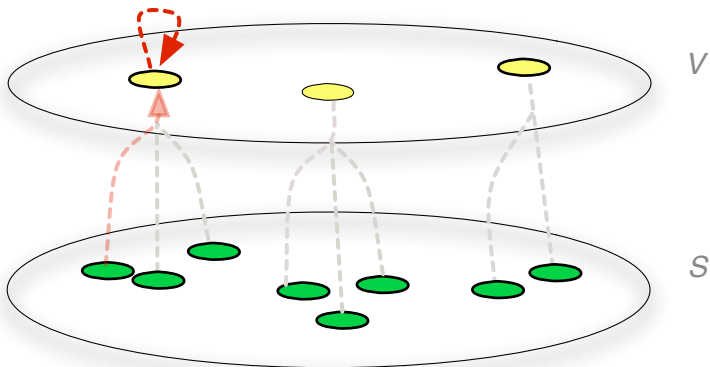
put (get s) s = s



Stability (GETPUT)

Principle: If the view does not change, then neither should the source.

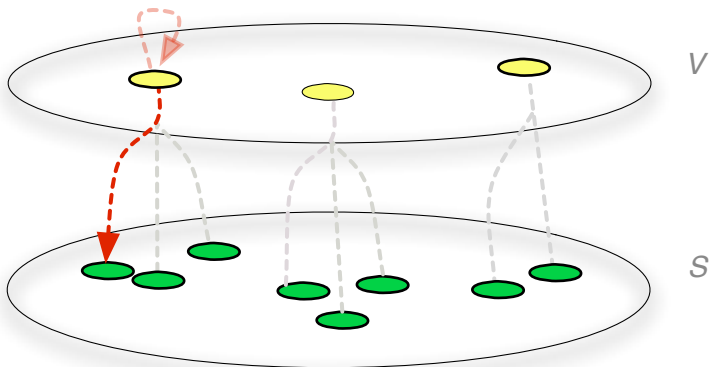
put (get s) s = s



Stability (GETPUT)

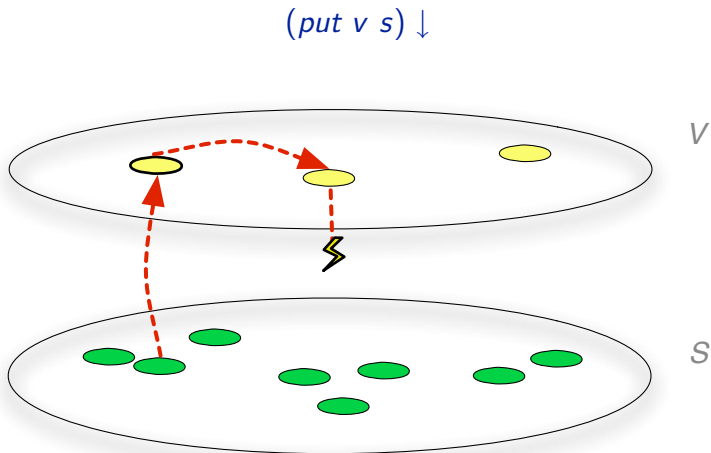
Principle: If the view does not change, then neither should the source.

put (get s) s = s



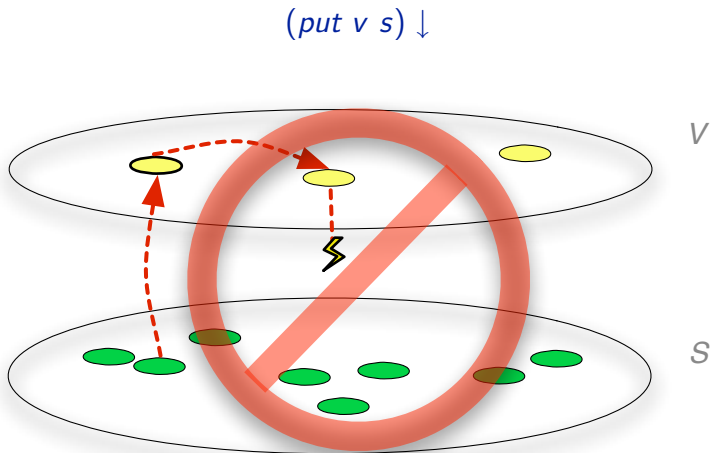
Totality

Principle: The *put* function should be able to do something with *any* view and *any* source.



Totality

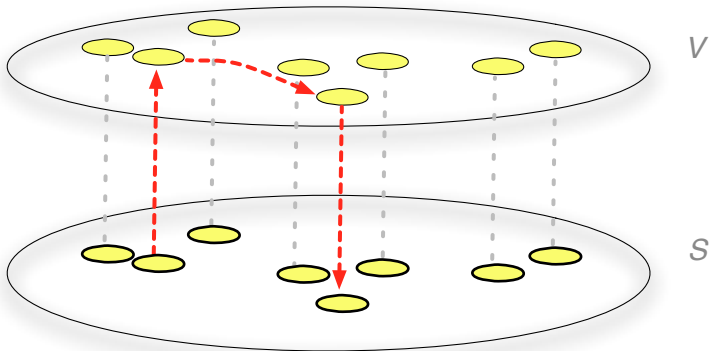
Principle: The *put* function should be able to do something with *any* view and *any* source.



Oblivious Lenses

Bijective transformations are a simple case... where *put* function doesn't use its source argument.

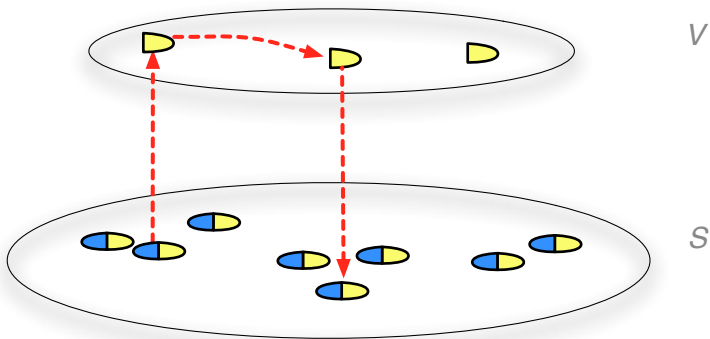
$$\text{Totality} + \text{Acceptability} + \text{Stability} + \text{put } v \ s = \text{put } v \ s'$$



Forgetfulness (PUTPUT)

Principle: The effect of two *puts* evaluated in sequence should be the same as just the second.

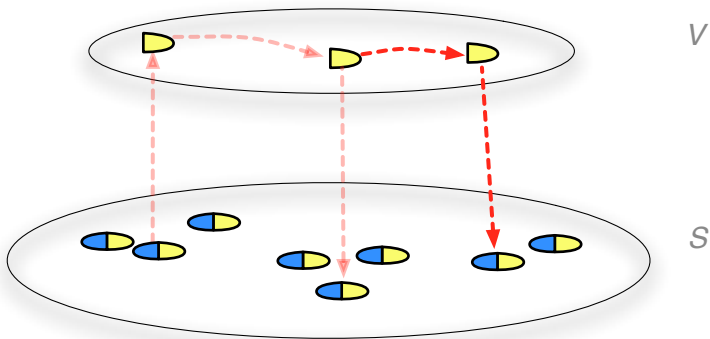
$$\text{put } v \ (\text{put } v' \ s) = \text{put } v \ s$$



Forgetfulness (PUTPUT)

Principle: The effect of two *puts* evaluated in sequence should be the same as just the second.

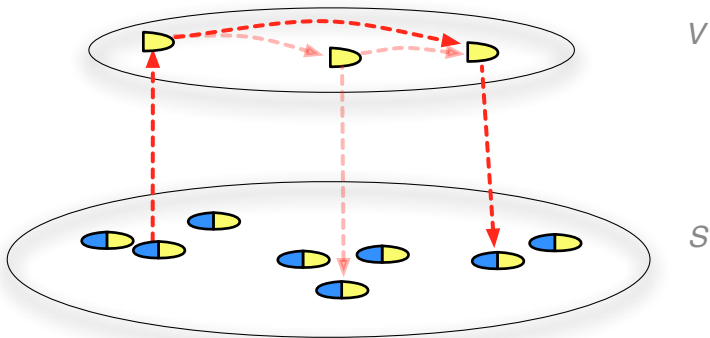
$$\text{put } v \ (\text{put } v' \ s) = \text{put } v \ s$$



Forgetfulness (PUTPUT)

Principle: The effect of two *puts* evaluated in sequence should be the same as just the second.

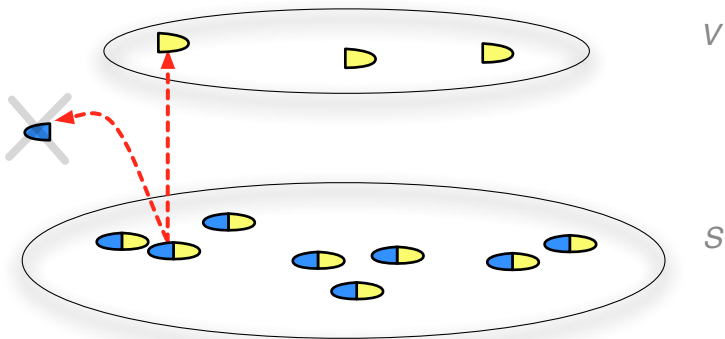
$$\text{put } v \ (\text{put } v' \ s) = \text{put } v \ s$$



Forgetfulness (PUTPUT)

Principle: The effect of two *puts* evaluated in sequence should be the same as just the second.

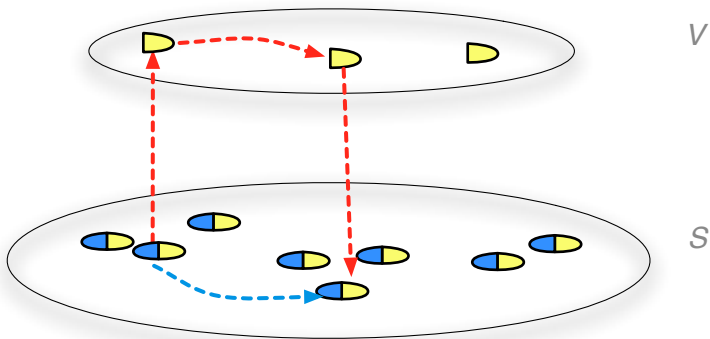
$$\text{put } v \ (\text{put } v' \ s) = \text{put } v \ s$$
$$\exists U. S \simeq U \times V$$



Forgetfulness (PUTPUT)

Principle: The effect of two *puts* evaluated in sequence should be the same as just the second.

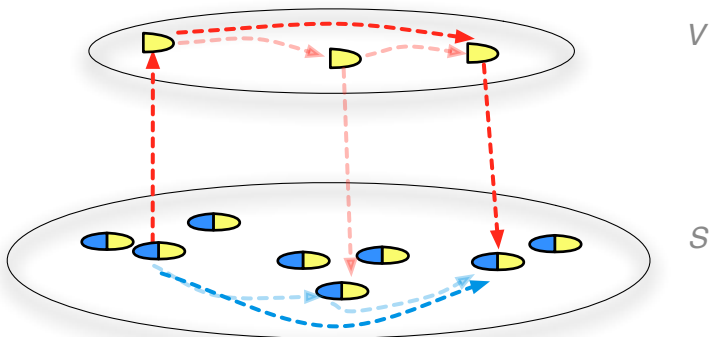
$$\text{put } v \ (\text{put } v' \ s) = \text{put } v \ s$$
$$\exists U. S \simeq U \times V$$



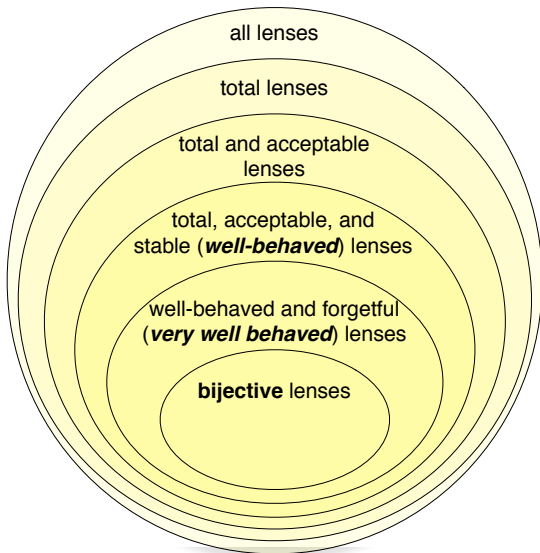
Forgetfulness (PUTPUT)

Principle: The effect of two *puts* evaluated in sequence should be the same as just the second.

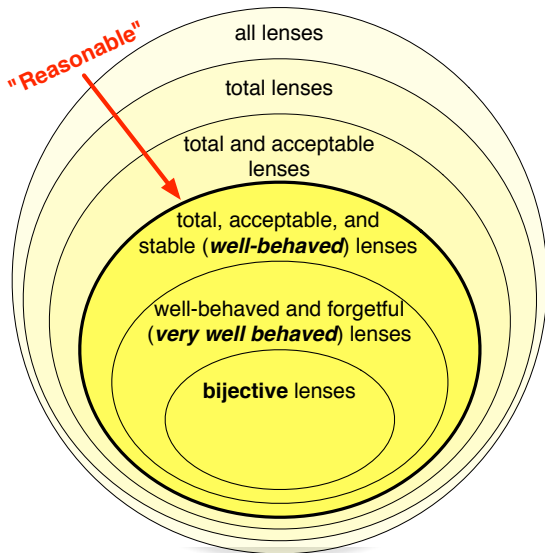
$$\text{put } v \ (\text{put } v' \ s) = \text{put } v \ s$$
$$\exists U. S \simeq U \times V$$



A Lens Bestiary



A Lens Bestiary



“Good men must not obey the laws too well.” –R W Emerson

Refined Laws

Quotient Lenses: only obey PUTGET and GETPUT up to specified equivalence relations on the source and view.

- ▶ Used in lenses that handle “ignorable” data.

Quasi-Oblivious Lenses: *put* is only oblivious on a portion of the source (specified by an equivalence relation).

- ▶ Used to ensure resourcefulness for lenses on ordered data.

Pretty Very Well-Behaved Lenses: the effect of two *puts* evaluated in sequence is the same as just the second. on a portion of the source (specified by an equivalence relation).

- ▶ Used to ensure integrity of source data.